

C++ Syntax Practice

FRE6883 – Financial Computation

Recitation 1 – Deck 4

One exercise at a time.

Exercise 1: predict the output

Basic

Core

Inside `main()`; `#include <iostream>` is assumed.

```
int x = 5;
double y = x / 2;

std::cout << y << std::endl;
```

1. What is printed, and why?
2. What type does the expression `x / 2` have?
3. Change one literal so that the program prints 2.5.

Solution 1: the division happens first

Basic – Solution

```
int x = 5;  
double y = x / 2;  
std::cout << y << std::endl;
```

Original output

2

```
int x = 5;  
double y = x / 2.0;  
std::cout << y << std::endl;
```

With a double operand

2.5

Both operands are int in `x / 2`. Integer division produces 2; assigning that result to a double gives 2.0.

Using 2.0 makes the division floating point. Default cout formatting prints 2.0 as 2.

Exercise 2: find the type problem

Basic

Optional reserve

Inside `main()`; `#include <iostream>` is assumed.

```
int stockPrice = 106.75;  
double rate = 3 / 100;
```

```
std::cout << stockPrice << " " << rate;
```

1. Is this valid C++? What value is stored in each variable?
2. Repair both initializations to preserve the intended values.
3. Which type suits a stock price, and which suits a count of steps?

Solution 2: choose the type and the expression

Basic – Solution

```
double stockPrice = 106.75;  
double rate = 3.0 / 100.0;  
int steps = 5;
```

Original output: 106 0

`int stockPrice = 106.75;` is valid, but may produce a compiler warning. Conversion to `int` discards the fractional part.

`3 / 100` is integer division, so `rate` receives `0.0`.

Use double for decimal quantities and int for counts.

The corrected rate is `0.03`; changing only its destination type cannot fix the division.

Exercise 3: complete the call payoff

Basic

Core

Inside `main()`; `#include <iostream>` is assumed. The user enters a number.

```
double S = 0.0, K = 100.0;
std::cin >> S;
double payoff = 0.0;
if (S > K)
{
    // complete this line
}
else
{
    // complete this line
}
std::cout << payoff;
```

1. Complete the European call payoff.
2. What is printed for inputs 90, 100, and 106?
3. Which path is taken when `S == K`?

Solution 3: one condition, two paths

Basic – Solution

```
if (S > K)
{
    payoff = S - K;
}
else
{
    payoff = 0.0;
}
std::cout << payoff;
```

Check the boundary

Input S	Output
90	0
100	0
106	6

$S > K$ is a comparison. Only one path executes.

At equality, the condition is false, so else assigns zero.

`std::cin >> S` reads input; `payoff = S - K` assigns the computed value.

Exercise 4: complete the validation

Basic

Core

Inside `main()`; S_0 , U , D are finite doubles. Assume `#include <iostream>`.

```
if (S0 <= 0.0 ___ U <= D ___ D <= 0.0)
{
    std::cout << "Invalid parameters";
}
```

1. Fill the blanks to validate $S_0 > 0$ and $U > D > 0$.
2. Which inputs below are rejected? Explain each decision.
3. Write one condition that is true exactly when these inputs are valid.

Case	S_0	U	D
A	100.0	1.2	0.8
B	100.0	0.9	1.1
C	0.0	1.2	0.8

Solution 4: reject any invalid component

Basic – Solution

```
if (S0 <= 0.0 || U <= D || D <= 0.0)
{
    std::cout << "Invalid parameters";
}
```

|| means OR; && means AND

An OR condition is true when at least one component is true.

The valid-input condition requires **all** three checks:

$S_0 > 0.0 \ \&\& \ U > D \ \&\& \ D > 0.0$

A passes. B fails because $U \leq D$. C fails because $S_0 \leq 0.0$.

These are the simplified checks $S_0 > 0$ and $U > D > 0$; the full model also checks R.

Exercise 5: trace a for loop

Basic

Optional reserve

Inside `main()`; `#include <iostream>` is assumed.

```
for (int i = 0; i < 5; i++)  
{  
    std::cout << i << " ";  
}
```

1. What is printed? How many times does the body execute?
2. What changes if the condition becomes `i <= 5`?
3. Rewrite the loop to print the integers from 4 down to 0.

Solution 5: initialize, test, execute, update

Basic – Solution

Original loop

0 1 2 3 4

5 iterations. At `i == 5`, the test fails.

With `i <= 5`

0 1 2 3 4 5

6 iterations. Equality is included.

```
for (int i = 4; i >= 0; i--)  
{  
    std::cout << i << " ";  
}
```

4 3 2 1 0

`i++` adds one; `i--` subtracts one.

A for loop tests its condition before each execution of the body.

Each example also prints one space after its final number.

Exercise 6: find the indexing bug

Basic

Core

Inside `main()`. Diagnose this code on paper before running it.

```
double prices[5];  
  
for (int i = 0; i <= 5; i++)  
{  
    prices[i] = 100.0;  
}
```

1. What is wrong, and at which iteration does it happen?
2. Which indices belong to this array?
3. Correct the loop so that every element is assigned exactly once.

Solution 6: the last index is size minus one

Basic – Solution

```
double prices[5];  
  
for (int i = 0; i < 5; i++)  
{  
    prices[i] = 100.0;  
}
```

Raw C++ arrays provide **no automatic bounds checking**. An invalid access has undefined behavior; a crash is not guaranteed.

The corrected loop assigns every element before it is read.

Valid indices: 0, 1, 2, 3, 4

Use $i < 5$.

The original sixth iteration writes to `prices[5]`, outside the array.

Exercise 7: visit the tree nodes

Intermediate

Core

Inside main(); #include <iostream> is assumed.

```
const int N = 3;
for (int n = 0; n <= N; n++)
{
    for (int i = 0; i <= n; i++)
    {
        std::cout << "(" << n << ", "
                    << i << ") ";
    }
}
```

1. How many times is the inner body executed in total?
2. List the pairs (n,i) in execution order.
3. Why is this loop structure natural for a binomial tree?

Solution 7: the inner loop restarts on each row

Intermediate – Solution

n	Pairs visited, in order	Executions
0	(0,0)	1
1	(1,0), (1,1)	2
2	(2,0), (2,1), (2,2)	3
3	(3,0), (3,1), (3,2), (3,3)	4

Total: $1 + 2 + 3 + 4 = 10$ executions

At time step n , the tree has $n + 1$ nodes, indexed from 0 through n . The inner loop initializes $i = 0$ again for each new n .

n selects the time step; i selects a node within that time step.

Exercise 8: complete and organize a function

Intermediate

Core

Use the course relation $S(n, i) = S_0 U^i D^{n-i}$; assume $0 \leq i \leq n$.

```
double CalculateAssetPrice(double S0, double U,  
                           double D, int n, int i)  
{  
    // complete  
}
```

1. Complete the function body.
2. Identify the return type and the type of each parameter.
3. Write its declaration in `BinomialTreeModel.h`, inside namespace `fre`.
4. Where does the definition belong, and how does `main()` call it?

Solution 8: a declaration promises; a definition implements

Intermediate – Solution

BinomialTreeModel.h

```
#pragma once
namespace fre
{
    double CalculateAssetPrice(
        double S0, double U, double D,
        int n, int i);
}
```

Main.cpp: include the header, then call:

```
fre::CalculateAssetPrice(
    100.0, 1.2, 0.8, 2, 1);
```

double: result, S0, U, D.

int: n, i. **fre::** selects the namespace.

The header declares; the function body defines. Compile both .cpp files together.

BinomialTreeModel.cpp

```
#include "BinomialTreeModel.h"
#include <cmath>
namespace fre
{
    double CalculateAssetPrice(
        double S0, double U, double D,
        int n, int i)
    {
        return S0 * std::pow(U, i)
            * std::pow(D, n - i);
    }
}
```

Exercise 9: trace a function call

Intermediate

Core

Complete program; `#include <iostream>` is assumed.

```
void Foo(int a, int& b)
{
    a++;
    b++;
}

int main()
{
    int x = 1;
    int y = 1;
    Foo(x, y);
    std::cout << x << " " << y;
}
```

1. What is printed after `Foo(x, y)`?
2. Track `a`, `b`, `x`, and `y` through the call.
3. What changes if the first parameter becomes `int& a`?

Solution 9: a copy and an alias behave differently

Intermediate – Solution

Output: 1 2

`int` `a` receives a **copy** of `x`. Incrementing `a` changes only that copy.

`int&` `b` is an **alias** for `y`. Incrementing `b` changes `y` itself.

	On entry to Foo	After both increments
<code>x</code> in main	1	1
<code>a</code> , the copy	1	2
<code>y</code> , also named <code>b</code>	1	2

With `int&` `a`, both arguments are modified: the output becomes 2 2.

Optional extension: reading a vector type

Intermediate

Before Exercise 10C only

One new type name

`std::vector<double>` denotes a sequence of doubles. Its type comes from `<vector>`;
`std::` is the standard-library namespace.

```
#include <vector>
```

```
// Inside main():
```

```
std::vector<double> prices = {100.0, 102.0, 98.0};
```

`prices[0]`, `prices[1]`, and `prices[2]` name the three elements.

Exercise 10C only asks you to choose a parameter type. No vector implementation is required. Skip this extension if the type is new to the group.

Exercise 10: choose the parameter type

Intermediate

Core A/B; optional C

// A: square x; keep caller unchanged.

```
double Square( ??? x );
```

// B: read into the caller's variable.

```
void ReadPrice( ??? price );
```

// C (optional): mean of a big vector.

// Read only; assume it is nonempty.

```
double Mean( ??? prices );
```

Choose from:

double

double&

const double&

std::vector<double>

std::vector<double>&

const std::vector<double>&

1. Choose a suitable type for each parameter and explain why.
2. Write the one-line bodies of A and B, assuming valid numeric input.
3. Would another type compile but be less suitable for the stated task?

Solution 10: match the parameter to its job

Intermediate – Solution

```
double Square(double x)
{
    return x * x;
}
```

```
void ReadPrice(double& price)
{
    std::cin >> price;
}
```

```
double Mean(const std::vector<double>& prices); // optional C
```

const double& also works for A but is unnecessary here. C by value copies the vector; a non-const reference permits edits through the parameter.

A: pass a small read-only value by value.

Copying one double is inexpensive.

B: use a reference to modify the caller.

A value parameter would only update a copy.

C: use a const reference for a large read-only object.

Avoid copying the vector; forbid edits through this parameter.

Exercise 11: does this function compile?

Intermediate

Optional reserve

```
void Foo(const int& x)
{
    x++;
}
```

1. Does this compile? Explain the rule involved.
2. What does `const` guarantee through this parameter?
3. Change the parameter type to modify the caller; then give a type that modifies only a copy.

Solution 11: const prevents writes through x

Intermediate – Solution

The original function does not compile

`x++` attempts to modify an object through `const int& x`. The reference allows reading, but not modifying, that object through `x`.

Modify the caller

```
void Foo(int& x)
{
    x++;
}
```

Modify only a local copy

```
void Foo(int x)
{
    x++;
}
```

A `const` reference does not make the caller's variable permanently `const`. The caller can still modify its own non-`const` variable.

Exercise 12: read the backward loop

Intermediate

Core

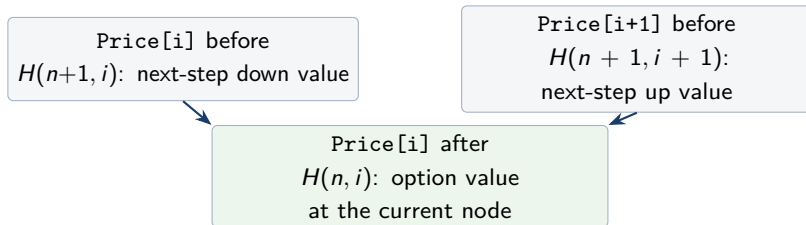
$N = 3$; Price has 4 elements holding terminal option payoffs. $R > 0$; $0 < q < 1$.

```
for (int n = N - 1; n >= 0; n--)  
{  
    for (int i = 0; i <= n; i++)  
    {  
        Price[i] =  
            (q * Price[i + 1]  
             + (1.0 - q) * Price[i]) / R;  
    }  
}
```

1. Why does n decrease, and which values does it take?
2. Why stop the inner loop at $i \leq n$?
3. What does $\text{Price}[i]$ represent just before and just after its assignment?
4. Which next-step values are used, and why is this backward induction?

Solution 12: replace the next row with the current row

Intermediate – Solution



For $N = 3$, n takes 2, 1, 0: move from known terminal payoffs back toward today.

Row n has **$n + 1$ nodes**, indexed 0 through n . At each assignment, $\text{Price}[i]$ and $\text{Price}[i+1]$ still hold next-step values.

The forward inner loop reads both values before overwriting $\text{Price}[i]$. After the final row, $\text{Price}[0]$ holds today's option value.

Exercise 13: find the largest price

Challenge

Optional

Assume $\text{size} > 0$, the array contains at least size finite values, and no edits are allowed.

```
double MaxPrice(const double prices[], int size);
```

1. Write the function definition using a loop.
2. Test it on {100.0, 120.0, 90.0, 110.0}, then on {106.75}.
3. Explain the indices your loop visits and why your initialization works.

Use variables, comparisons, indexing, and a loop; no library maximum algorithm.

Solution 13: carry the largest value seen so far

Challenge – Solution

```
double MaxPrice(const double prices[],
                int size)
{
    double largest = prices[0];
    for (int i = 1; i < size; i++)
    {
        if (prices[i] > largest)
        {
            largest = prices[i];
        }
    }
    return largest;
}
```

Start from a real element

largest is the maximum of all elements visited so far.

Results: 120.0 and 106.75.

With one element, the loop runs zero times and returns prices[0].

Assume size > 0; const prevents edits through prices.